Every equation in the Kalman filter code

and where it comes from

Robot firmware: `KalmanFilter`, `MotionEstimator`, and the calculations that feed them

Conventions. Heading and angular velocity are **clockwise positive** (radians). The time step is $dt = 0.02$ s (50 Hz control loop). The covariance matrix is written with these names:

$$P = \begin{pmatrix} p & c \\ c & s \end{pmatrix} \quad p = \text{estimateVariance}, \quad c = \text{crossCovariance}, \quad s = \text{sensorBiasVariance}$$

Part A: Inputs to the filter

A1. Converting raw gyro readings to rad/s

Code: `Accelerometer.cpp — readMotion()`

$$\omega_{\text{gyro}} = \frac{\text{raw} - \text{offset}}{131.2} \cdot \frac{\pi}{180}$$

Derivation. The sensor stores a signed 16-bit value ($-32768 \ldots +32767$) for a range of ± 250°/s:

$$\frac{32768}{250} \approx 131.07 \rightarrow \text{datasheet value } \; 131.2 \; \text{ LSB per } \; ^\circ/\text{s}$$

Dividing by 131.2 gives °/s; multiplying by $\pi/180$ converts to rad/s, because $180° = \pi$ rad.

The gyroscope's Z axis points up, so it reads **counter-clockwise** as positive. `MotorManager` multiplies by `gyroDirection = -1` to match the clockwise-positive heading:

$$\omega = \text{gyroDirection} \cdot \omega_{\text{gyro}} = -\omega_{\text{gyro}}$$

A2. Converting raw accelerometer readings to m/s²

$$a_{\text{forward}} = \text{sign} \cdot \frac{\text{raw} - \text{offset}}{16384/9.81}$$

Derivation. With a range of $\pm 2g$: $32768/2 = 16384$ LSB per g. Dividing by 16384 gives g; multiplying by 9.81 gives m/s². The code folds both into the single divisor $16384/9.81$.

A3. Startup offset calibration (arithmetic mean)

Code: `Accelerometer.cpp — calibrateGyroscopeOffset()`

$$\text{offset} = \frac{1}{N}\sum_{i=1}^{N} \text{raw}_i, \quad N = 300$$

Derivation. While the robot is still, the true rate is 0, so each reading is offset + noise. The noise averages to zero, so the mean is the best estimate of the offset. Averaging N samples shrinks the noise's standard deviation by $\sqrt{N}$: 300 samples are about 17 × less noisy than one.

A4. Encoder heading for a differential drive

Code: `MotorManager.cpp — updateHeading()`

$$\theta_{\text{enc}} = \theta_{\text{start}} + \frac{d_L - d_R}{W}, \quad W = \text{wheelTrack} = 0.145 \;\; \text{m}$$

Derivation. When the robot turns clockwise by $\Delta\theta$ about some point, each wheel moves along a circular arc (arc length = radius × angle). Turning clockwise, the left wheel is on the outside, and the two radii differ by exactly W:

$$d_L = \left(r + \frac{W}{2}\right)\Delta\theta, \quad d_R = \left(r - \frac{W}{2}\right)\Delta\theta$$

$$d_L - d_R = W\Delta\theta \quad \Rightarrow \quad \Delta\theta = \frac{d_L - d_R}{W}$$

The turning radius r cancels out, so the formula holds for any turn; turning in place ($r = 0$) is one case. `leftDirection` and `rightDirection` give the wheel distances their signs.

A5. Encoder velocity

Code: `MotorManager.cpp — moveForDistance()`

$$v_{\text{enc}} = \frac{(\Delta L + \Delta R)/2}{dt}$$

Derivation. The center of the axle is halfway between the wheels, so its distance is the average of the two wheel distances. Velocity is distance over time, using a finite difference over one 20 ms step.

Part B: The Kalman filter

B1. The physical model

Code: `KalmanFilter.hpp — header comment`

The IMU measures a **rate**, the time derivative of the estimated value, plus a constant bias:

$$\text{imuRate} = \frac{dx}{dt} + b + \text{noise} \quad \Rightarrow \quad \frac{dx}{dt} = \text{imuRate} - b$$

Integrating over one small step dt (Euler integration) gives:

$$\begin{aligned} x_{k+1} &= x_k + (\text{imuRate} - b_k)dt + w_x \\ b_{k+1} &= b_k + w_b \end{aligned}$$

where w_x, w_b are random disturbances: vibration, integration error, slow bias drift.

Matrix form. With state $\boldsymbol{x} = \begin{pmatrix} x \\ b \end{pmatrix}$ and input $u = \text{imuRate}$:

$$\boldsymbol{x}_{k+1} = F\boldsymbol{x}_k + Gu + \boldsymbol{w}, \quad F = \begin{pmatrix} 1 & -dt \\ 0 & 1 \end{pmatrix}, \quad G = \begin{pmatrix} dt \\ 0 \end{pmatrix}, \quad Q = \text{cov}(\boldsymbol{w}) = \begin{pmatrix} q & 0 \\ 0 & q_b \end{pmatrix}$$

The $-dt$ in F comes directly from the $-b \cdot dt$ term above.

B2. State prediction

Code: `KalmanFilter.cpp — update(), line 14`

```
estimate += (imuRate - sensorBias) * dt;
```

This is the model with the noise left out: noise can't be predicted and its expected value is 0. The bias row of F is $(0\ 1)$, so the predicted bias is unchanged and needs no code.

B3. Covariance prediction: $P = FPF^T + Q$

Code: `KalmanFilter.cpp — update(), lines 16–19`

Why this form. P is the covariance of the estimation error $\boldsymbol{e} = \boldsymbol{x} - \hat{\boldsymbol{x}}$ (true minus estimate): $P = E[\boldsymbol{e}\boldsymbol{e}^T]$. The error moves forward like the state, $\boldsymbol{e}_{k+1} = F\boldsymbol{e}_k + \boldsymbol{w}$, so

$$P_{k+1} = E[(F\boldsymbol{e} + \boldsymbol{w})(F\boldsymbol{e} + \boldsymbol{w})^T] = FE[\boldsymbol{e}\boldsymbol{e}^T]F^T + E[\boldsymbol{w}\boldsymbol{w}^T] = FPF^T + Q$$

The cross terms vanish because $\boldsymbol{w}$ is independent of $\boldsymbol{e}$.

Multiplying it out:

$$FP = \begin{pmatrix} 1 & -dt \\ 0 & 1 \end{pmatrix}\begin{pmatrix} p & c \\ c & s \end{pmatrix} = \begin{pmatrix} p - dt \cdot c & c - dt \cdot s \\ c & s \end{pmatrix}$$

$$FPF^T = \begin{pmatrix} p - dt\cdot c & c - dt\cdot s \\ c & s \end{pmatrix}\begin{pmatrix} 1 & 0 \\ -dt & 1 \end{pmatrix} = \begin{pmatrix} p - 2dt\cdot c + dt^2 s & c - dt\cdot s \\ c - dt\cdot s & s \end{pmatrix}$$

Adding Q gives exactly the three lines of code:

$$\begin{aligned} p &\leftarrow p - 2dt\cdot c + dt^2 s + q \\ c &\leftarrow c - dt\cdot s \\ s &\leftarrow s + q_b \end{aligned}$$

```
estimateVariance   += -2.0f * dt * crossCovariance + dt * dt * sensorBiasVariance +
estimateProcessNoise;
crossCovariance    -= dt * sensorBiasVariance;
sensorBiasVariance += sensorBiasProcessNoise;
```

The **order** matters and is correct: p needs the old c and s, so it goes first; c needs the old s; s goes last.

Meaning: p grows on every step, so prediction makes the filter less certain. c becomes negative, which records that "if my bias is too low, my estimate is too high".

B4. The measurement model

$$z = H\boldsymbol{x} + v, \quad H = (1\ \ 0), \quad R = \mathrm{var}(v) = \mathrm{measurementNoise}$$

The encoder measures the estimated value directly, with no bias, so H picks out the first state only.

B5. Innovation and its variance

Code: `KalmanFilter.cpp — update(), lines 22–23`

$$y = z - H\hat{\boldsymbol{x}} = \mathrm{encoder} - \mathrm{estimate}$$

$$S = HPH^T + R = p + R$$

Derivation of S. $y = H\boldsymbol{e} + v$. Its variance is HPH^T (from the state error) plus R (from sensor noise), since the two are independent. With $H = (1\,0)$, HPH^T picks out the top-left element p.

B6. The Kalman gain

Code: `KalmanFilter.cpp — update(), lines 24–25`

$$K = \frac{PH^T}{S} = \frac{1}{S}\begin{pmatrix} p \\ c \end{pmatrix} \quad \Rightarrow \quad K_1 = \mathrm{estimateGain} = \frac{p}{S}, \quad K_2 = \mathrm{sensorBiasGain} = \frac{c}{S}$$

1-D intuition. Suppose there are two independent guesses of the same value: the prediction $\hat{x}$ with variance p, and the measurement z with variance R. The minimum-variance combination is the inverse-variance weighted average:

$$x_{\mathrm{new}} = \frac{R\hat{x} + pz}{p + R} = \hat{x} + \frac{p}{p+R}(z - \hat{x}) = \hat{x} + Ky, \quad K = \frac{p}{p+R}$$

So K is "my uncertainty ÷ total uncertainty". $K \approx 0$ means trust the prediction; $K \approx 1$ means trust the encoder.

General derivation. With the update $\hat{\boldsymbol{x}}^+ = \hat{\boldsymbol{x}} + Ky$, the new error is $\boldsymbol{e}^+ = (I - KH)\boldsymbol{e} - Kv$, so

$$P^+ = (I - KH)P(I - KH)^T + KRK^T$$

Choose K to minimise the total error variance $\mathrm{tr}(P^+)$:

$$\frac{\partial\ \mathrm{tr}(P^+)}{\partial K} = -2PH^T + 2K(HPH^T + R) = 0 \quad \Rightarrow \quad K = PH^T S^{-1}$$

Why the bias is learned at all: the encoder never measures the bias, but $K_2 = c/S \neq 0$ because the prediction step linked estimate errors to bias errors through c. If $c = 0$ (as in the heading filter), $K_2 = 0$ and the bias never changes.

B7. State update

Code: `KalmanFilter.cpp — update(), lines 27–28`

$$\hat{\boldsymbol{x}} \leftarrow \hat{\boldsymbol{x}} + Ky \quad \Rightarrow \quad \text{estimate} \mathrel{+}= \frac{p}{S} \cdot y, \quad \text{sensorBias} \mathrel{+}= \frac{c}{S} \cdot y$$

Sign check (velocity filter): $c < 0$. If $y < 0$ (encoder slower than prediction), then $(c/S) \cdot y > 0$, so the bias increases, the next prediction subtracts more, and the error shrinks. That is the correct response.

B8. Covariance update: $P = (I - KH)P$

Code: `KalmanFilter.cpp — update(), lines 30–33`

Derivation. Substituting the optimal K into the full P^+ formula above, the KSK^T term cancels part of the expansion, leaving $P^+ = (I - KH)P$.

Multiplying it out:

$$I - KH = \begin{pmatrix} 1 - K_1 & 0 \\ -K_2 & 1 \end{pmatrix}, \quad (I - KH)P = \begin{pmatrix} (1 - K_1)p & (1 - K_1)c \\ c - K_2 p & s - K_2 c \end{pmatrix}$$

Element	Math	Code
top-left	$p \leftarrow p - K_1 p$	`estimateVariance -= estimateGain * estimateVariance`
top-right	$c \leftarrow c - K_1 c$	`crossCovariance -= estimateGain * crossCovariance`
bottom-right	$s \leftarrow s - K_2 c$	`sensorBiasVariance -= sensorBiasGain * crossCovariance`

The bottom-left element $c - K_2 p$ equals the top-right because $K_2 p = cp/S = K_1 c$, so P stays symmetric and storing one c is enough. The order is correct again: s uses the old c; c and p use their own old values.

Meaning: every term is multiplied by something below 1, so a measurement always reduces uncertainty.

B9. Reset

Code: `KalmanFilter.cpp — reset()`

$$x = x_{\text{start}}, \quad p = 0, \quad c = 0 \quad (\text{s and the bias are kept})$$

This is a modelling choice, not a derived formula: "at the start of a move I know exactly where I am relative to the start" gives $p = 0$. Knowledge of the bias (s) carries over between moves.

Part C: MotionEstimator

The estimator uses two independent copies of the filter. They can be separate because the robot either drives straight or turns in place, never both at once.

	Heading filter	Velocity filter
IMU input	gyro Z (rad/s, clockwise +)	forward accel (m/s²)
Encoder input	$\theta_{\text{start}} + (d_L - d_R)/W$	average wheel speed
Estimate	heading (rad, clockwise +)	forward velocity (m/s)
Bias learned by filter	no ($q_b = 0$, $s_0 = 0$)	yes
Q per step	10^{-6}	10^{-4}

R	0.25 (encoder trusted very little)	0.0004 (encoder trusted a lot)

C1. Gyro bias calibration (exponential moving average)

Code: `MotionEstimator.cpp — calibrateGyroBias()`

$$b \leftarrow b + \alpha(z - b) = (1-\alpha)b + \alpha z, \quad \alpha = 0.1$$

Derivation. While the robot is still, the true rate is 0, so the reading z is bias + noise. This is a Kalman update for a constant with a **fixed** gain $K = \alpha$ instead of a calculated one. Unrolling it shows old information fades geometrically:

$$b_n = (1-\alpha)^n b_0 + \alpha \sum_{i=1}^{n} (1-\alpha)^{n-i} z_i$$

With $n = 15$ samples (before each move), the starting value keeps weight $0.9^{15} \approx 0.21$, so each move gets about 79% of the way to the true bias.

C2. Why the heading filter doesn't learn the bias

The heading filter is built with $q_b = 0$ and $s_0 = 0$. From the covariance prediction, $c \leftarrow c - dt \cdot s$ stays 0 forever; from the gain, $K_2 = c/S = 0$. The bias only changes through the calibration above.

Why this was chosen: a mismatch in wheel diameters ($d_L \neq d_R$ while driving straight) makes the encoder heading grow at a constant wrong rate. That looks exactly like a constant gyro bias, so the filter cannot tell the two apart while driving.

C3. Bias-corrected turn rate

Code: `MotorManager.cpp — headingRate()`

$$\omega = \text{gyroDirection} \cdot \text{gyroZ} - b$$

The same $\text{imuRate} - b$ as in the physical model, used as the D term of the heading controller and for the rotation braking lookahead.

C4. Why the velocity filter is used only for straight driving

The IMU is not at the center of rotation. A sensor at distance r from the center feels a centripetal acceleration while turning:

$$a_c = \omega^2 r$$

The filter would wrongly learn that as accelerometer bias.

C5. What the noise constants mean

Q and R are **variances**, the square of a standard deviation σ:

Constant	Value	$\sigma = \sqrt{\text{value}}$	Meaning
`r_angle`	0.25 rad²	0.5 rad ≈ 29°	encoder heading treated as very unreliable
`q_angle`	10^{-6} rad²/step	0.001 rad ≈ 0.06°/step	gyro integration is trusted
`r_velocity`	$4 \cdot 10^{-4}$ (m/s)²	0.02 m/s	encoder velocity is trusted
`q_velocity`	10^{-4} (m/s)²/step	0.01 m/s/step	accelerometer is noisy (vibration)
`p_accelBias`	0.01 (m/s²)²	0.1 m/s²	≈ 0.6° tilt: $9.81 \sin(0.6°) \approx 0.1$

C6. Steady-state gain (why the heading gain stays tiny)

If the heading filter ran forever with $s = c = 0$, the variance would settle where growth from prediction equals shrinkage from the update:

$$p = \frac{(p+q)R}{p+q+R} \quad \overset{q \ll R}{\Rightarrow} \quad p_\infty \approx \sqrt{qR}$$

$$K_\infty \approx \sqrt{\frac{q}{R}} = \sqrt{\frac{10^{-6}}{0.25}} = 0.002$$

Because `reset()` sets $p = 0$ at every move, K grows roughly as nq/R and never even nears 0.002 during a normal move. In practice a heading reading is about 99.9% gyro.